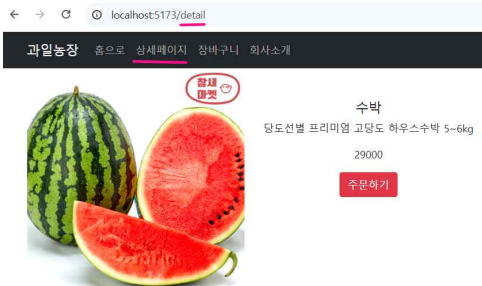
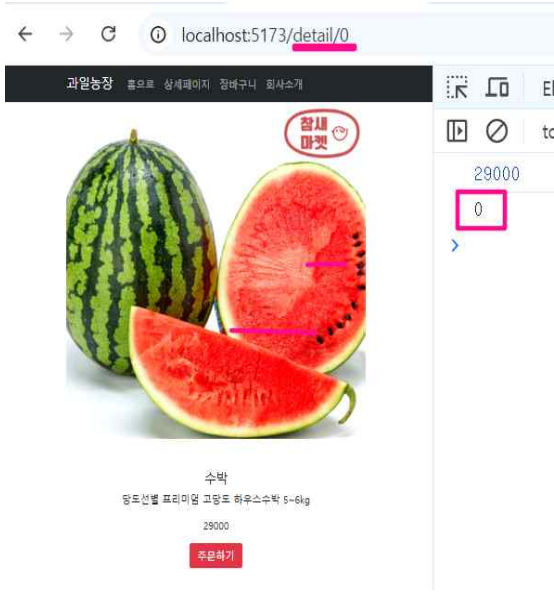
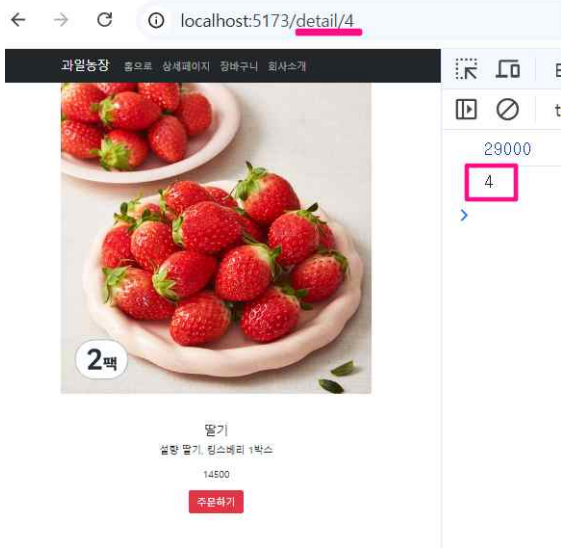


6. React Router의 URL 파라미터(param) 기능을 이용해 상세 페이지 생성

순서	작업환경	명령어
1	App.jsx 안에	<code><Route path="/detail" element={<Detail fruit={fruit}/>} /></code>
2	src폴더 > components > Detail.jsx 수정	<pre>import React from 'react'; function Detail(props) { return (<div className="container"> <div className="row"> <div className="col-md-6"> </div> <div className="col-md-6"> <h5 className="pt-5">{props.fruit[0].title}</h5> <p>{props.fruit[0].content}</p> <p>{props.fruit[0].price}</p> <button className="btn btn-danger">주문하기</button> </div> </div> </div>) } export default Detail;</pre>

브라우저에서 확인
http://localhost:5173/detail



		브라우저에서 확인, URL 작성, 크롬창에서 콘솔(F12), paramId 값 확인 http://localhost:5173/detail/0 http://localhost:5173/detail/4  
	순서 라우터는 경로를 미리 정해두는 작업이고, 네비게이션 링크는 그 경로로 이동	1 라우팅 경로(주소)를 먼저 설정한다. 예: Route path="/detail/:paramId" 와 같이 동적 경로를 React Router에 등록한다. :paramId는 변수처럼 바뀌는 값 (params) 이다. 2 링크(NavLink 또는 Link 등)를 통해 이동할 수 있도록 한다. 예: <Link to="/detail/1">1번 상세보기</Link>

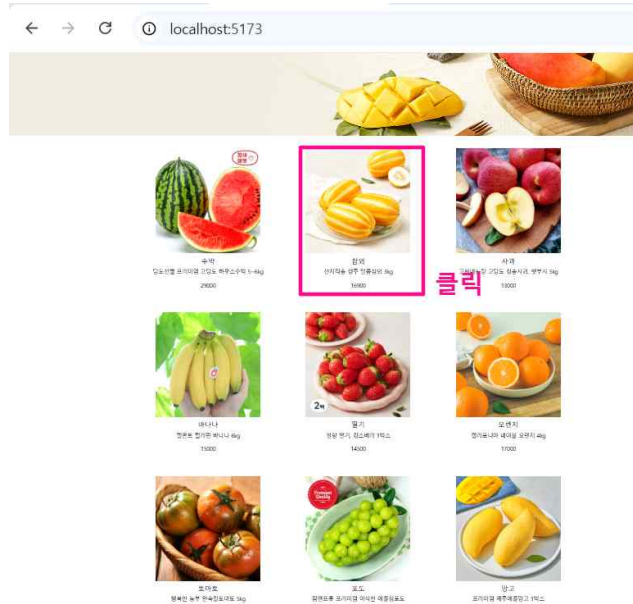
7. 메인페이지에서 상품 상세페이지 네비게이션 (링크 연결)

순서	작업환경	명령어
1	App.jsx slider 아래쪽에 상품 컴포넌트 수정 및 확인 (스타일 추가)	<pre> <div className="container" style={{marginTop:'30px'}}> <div className="row"> { fruit.map((ele,i)=>{ return(<Product fruit={fruit[i]} i={i} key={data[i].id} />) }) } </div> </div> </pre>
2	Product.jsx 안에 파라미터 라우터로 상품페이지 넘어가기	<pre> import React from "react"; import { useNavigate } from "react-router-dom"; import { Nav } from "react-bootstrap"; const Product = (props) => { const { id, title, price, imgUrl, content } = props.fruit; let navigate = useNavigate(); return (<div className="col-md-4" style={{ marginBottom: "50px" }}> <Nav.Link onClick={() => { navigate("/detail/" + (id-1)) }} className="c1"> <h5 style={{ marginTop: "10px" }}>{title}</h5> <p>{content}</p> {price} </Nav.Link> </div>); }; export default Product; </pre>
	index.css	<pre> .c1 {text-decoration: none; color:#000; text-align: center;} .c1:hover {color: green;} </pre>

브라우저에서 확인

메인에서 상품 클릭하면 참외-> 참외 상품으로 잘 가는지 확인, 다른 상품도 맞게 잘 가는지 체크

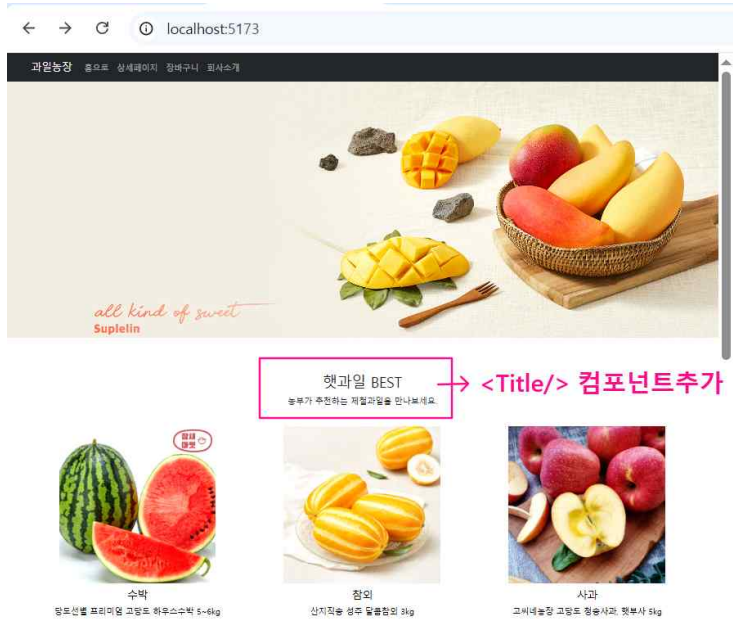
http://localhost:5173/

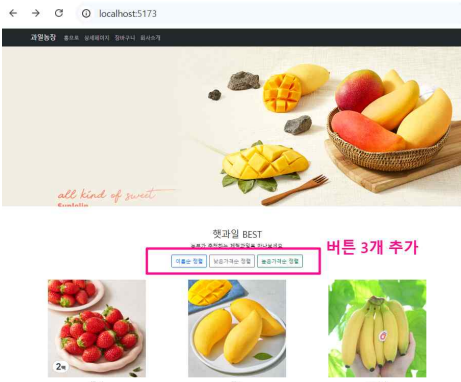
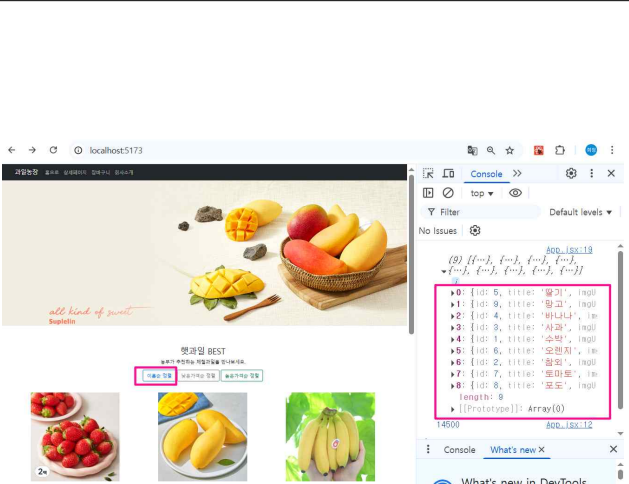


http://localhost:5173/detail/1



8. title과 버튼 배치

순서	작업환경	명령어	브라우저에서 확인
1	src폴더 > components > >Title.jsx 만들기	<pre>import React from "react"; const Title = () => { let csst1 = { marginTop: "70px", textAlign: "center", }; return (<> <h3 style={csst1}>햇과일 BEST</h3> <p style={{ textAlign: "center" }}> {" "} 농부가 추천하는 제철과일을 만나보세요. </p> </>); }; export default Title;</pre>	
	App.jsx 안에 slide 아래 배치	상단	import Title from './components/Title'
		하단	<Title/>

순서	작업환경	명령어
2	App.jsx 상단에 확인 (둘다 있음) title 아래 버튼 3개 만들기 App.jsx 안에 <Title/> 아래 버튼 넣기	App.jsx 상단에 확인 (둘다 있음) <pre>import { useState } from 'react'; import { Button, Navbar, Container, Nav } from 'react-bootstrap'</pre> App.jsx 리턴 위에 <pre>let [fruit, setFruit] = useState(data); const sortByName = () => { let sortedFruit = [...fruit].sort((a, b) => (a.title > b.title ? 1 : -1)); setFruit(sortedFruit); console.log(sortedFruit); }; const sortByPriceLowToHigh = () => { let sortedFruit = [...fruit].sort((a, b) => a.price - b.price); setFruit(sortedFruit); console.log(sortedFruit); }; const sortByPriceHighToLow = () => { let sortedFruit = [...fruit].sort((a, b) => b.price - a.price); setFruit(sortedFruit); console.log(sortedFruit); };</pre>  

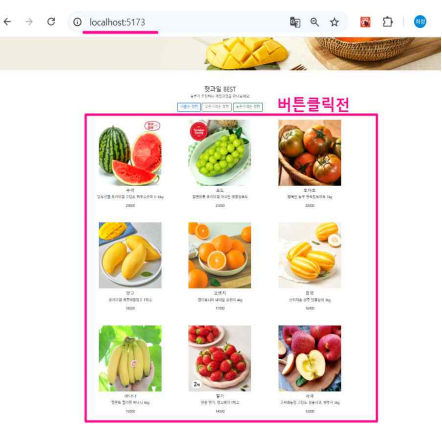
상품 정렬 후 링크가 올바르게 작동하도록 하기

순서	작업환경	명령어	
2-2	App.jsx 과일 9개 데이터 map 수정	<pre> <div className="container" style={{marginTop:'30px'}}> <div className="row"> {fruit.map((ele, i) => { return (<Product fruit={fruit[i]} i={i} key={fruit[i].id} />); })} </div> </div> </pre>	//sort된 fruit 배열에서 고유 id로 key 설정
	Product.jsx	<pre> import React from "react"; import { useNavigate } from "react-router-dom"; import { Nav } from "react-bootstrap"; const Products = (props) => { const { id, title, price, imgUrl, content } = props.fruit; let navigate = useNavigate(); return (<div className="col-md-4" style={{ marginBottom: "50px" }}> <Nav.Link onClick={() => { navigate("/detail/" + id) }} className="c1"> <h5 style={{ marginTop: "10px" }}>{title}</h5> <p>{content}</p> {price} </Nav.Link> </div>); }; export default Products; </pre>	// 해당 상품 클릭 시 상세 페이지로 이동

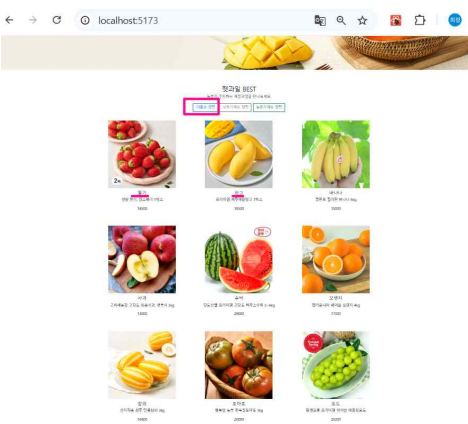
	Detail.jsx	<pre> import React from 'react'; import { useParams } from "react-router-dom"; function Detail(props) { let {paramId} = useParams(); console.log(paramId); //const { id, imgUrl, title, content, price } = props.fruit[paramId]; // props.fruit 배열에서 id가 paramId와 일치하는 상품을 찾아 item 변수에 저장 // paramId는 문자열이므로 parseInt를 사용해 숫자로 변환 let item = props.fruit.find(f => f.id === parseInt(paramId)); const { imgUrl, title, content, price } = item; return (<div className="container"> <div className="row"> <div className="col-md-6"> </div> <div className="col-md-6"> <h5 className="pt-5">{title}</h5> <p>{content}</p> <p>{price}</p> <button className="btn btn-danger">주문하기</button> </div> </div> </div>) } export default Detail; </pre>	유효성 체크란, 입력된 값이나 데이터가 올바르고 사용 가능한지 검사하는 과정 유효성 체크 내가 선택한 상품의 paramId와 fruit 배열 안의 id가 일치하는 과일 찾기 상품 유효성 체크는 Hook 호출 이후에 실행 hook(useState, useEffect, useParams)은 항상 같은 순서로 실행돼야 하므로 조건문이나 반복문 안이 아닌, 컴포넌트 최상단에서 먼저 호출 props.fruit 배열에서 id가 paramId와 일치하는 객체를 찾아 반환. .find()는 배열에서 조건을 만족하는 첫 번째 객체를 반환. item 객체에서 필요한 속성을 개별 변수로 분해
--	------------	---	--

브라우저에서 확인

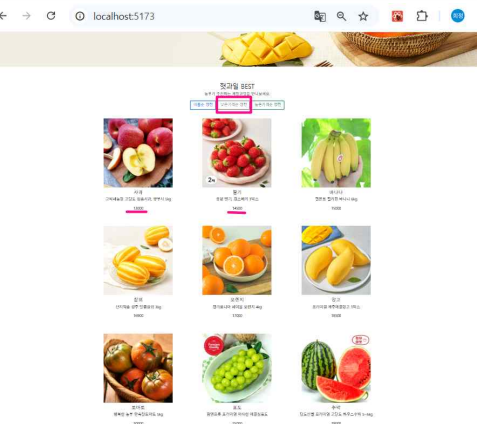
버튼 클릭전



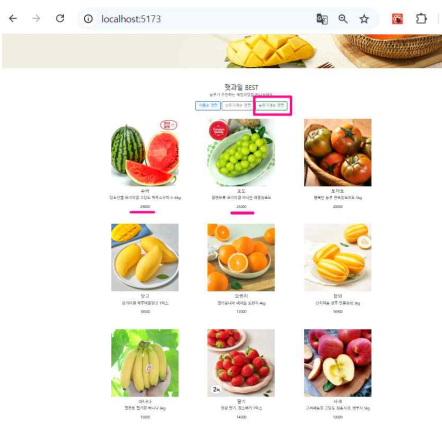
이름순 정렬



낮은 가격순 정렬

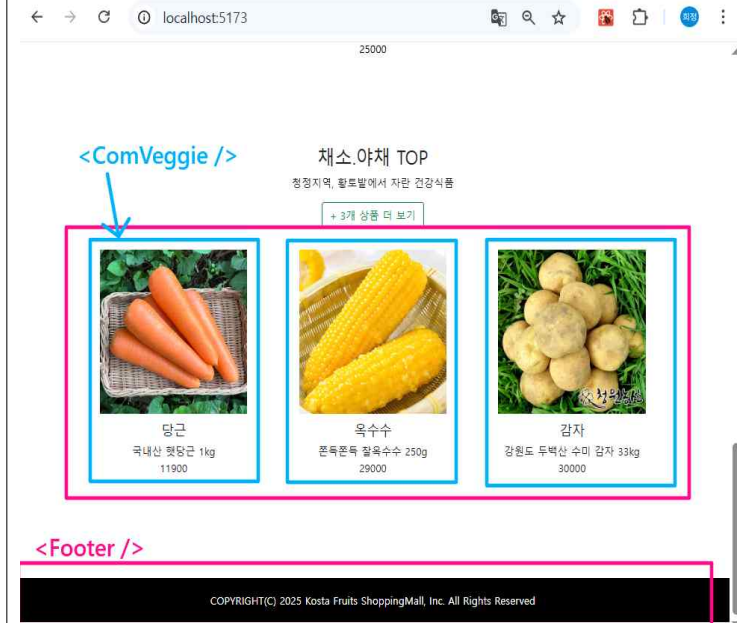


높은 가격순 정렬



순서	작업환경	명령어	브라우저 확인
3	title2만들기 src폴더 > components > > Title2.jsx 만들기	<pre>import React from "react"; const Title2 = () => { let csst1 = { marginTop: "70px", }; return (<> <h3 style={csst1}>채소.야채 TOP</h3> <p>청정지역, 황토밭에서 자란 건강식품</p> </>); }; export default Title2;</pre>	
	App.jsx 3개 상품 아래 <div className="container"></div> 아래 Product 위에 상품9개 밑에 배치	상단 <pre>import Title2 from './components/Title2'</pre> 하단 추가 <pre><div class="container"> <div class="row"> <div style={{ textAlign: "center" }}> <Title2 /> </div> </div> </div></pre>	
4	+3개 상품 더보기 버튼 만들기	App.js의 <Title2 />아래	<pre><Button variant="outline-success"> + 3개 상품 더 보기 </Button>{' '}</pre>

순서	작업환경	명령어
5	9개 상품 밑에 veggie 야채, 채소 카테고리 만들어야 하니까 준비	
6	1 야채 상품 3개 데이터 준비 src폴더 > db폴더 > veggie.js 만들기	<pre> let veggie = [{ id: 1, title: "당근", imgUrl: "img/veggie/veggie1.jpg", content: "국내산 햇당근 1kg", price: 11900, }, { id: 2, title: "옥수수", imgUrl: "img/veggie/veggie2.jpg", content: "쫄쫄득득 찰옥수수 250g", price: 29000, }, { id: 3, title: "감자", imgUrl: "img/veggie/veggie3.jpg", content: "강원도 두백산 수미 감자 33kg", price: 30000, },]; export default veggie; </pre>



10	footer만들기 src폴더 > components > > Footer.jsx만들기	<pre>import React from "react"; const Footer = () => { let foo = { color: "#fff", backgroundColor: "#000", padding: "20px 0px", marginTop: "80px", }; return (<> <p style={foo}>COPYRIGHT(C) 2025 Kosta Fruits ShoppingMall, Inc. All Rights Reserved</p> </>); }; export default Footer;</pre>	
	App.jsx 안에서 야채.상품 아래 맨 아래 배치	상단	import Footer from './components/Footer'
		하단	<Footer/>
11	라우터 확인 네비 링크 걸기 1 로고에 메인화면 2 상세페이지에 detail/1	<pre><Navbar.Brand onClick={()=>{ navigate('/')}}>과일농장</Navbar.Brand> <Nav className="me-auto"> <Nav.Link onClick={()=>{ navigate('/')}}>홈으로</Nav.Link> <Nav.Link onClick={()=>{ navigate('/detail/1')}}>상세페이지</Nav.Link> <Nav.Link onClick={() => { navigate('/cart')}}>장바구니</Nav.Link> <Nav.Link onClick={() => { navigate('/about')}}>회사소개</Nav.Link> </Nav></pre>	

9. JSON 파일 생성 후 Git 서버에 업로드 해두기

순서	작업환경	명령어
1	json파일 만들기	veggie2.json <pre>[{ "id" : 4, "title" : "고추", "imgUrl" : "img/veggie/veggie4.jpg", "content" : "혈당조절 청양고추, 300g", "price" : 21000 }, { "id" : 5, "title" : "밤", "imgUrl" : "img/veggie/veggie5.jpg", "content" : "아산울림 햇 약단밤", "price" : 14500 }, { "id" : 6, "title" : "고구마", "imgUrl" : "img/veggie/veggie6.jpg", "content" : "늘해랑 꿀고구마 10kg", "price" : 19800 }]</pre>
		veggie3.json <pre>[{ "id" : 7, "title" : "열무", "imgUrl" : "img/veggie/veggie7.jpg", "content" : "어린열무 싱싱 국내산 수확 4kg", "price" : 15000 },]</pre>



		<pre>{ "id" : 8, "title" : "무", "imgUrl" : "img/veggie/veggie8.jpg", "content" : "제주 세척 월동무", "price" : 21000 }, { "id" : 9, "title" : "오이", "imgUrl" : "img/veggie/veggie9.jpg", "content" : "경북상주 백다다기 흑침오이", "price" : 17800 }]</pre>
2	<u>github 에</u> <u>본인계정명.github.io</u> <u>저장소 생성</u>	github.com > 로그인 > your Repositories > 본인계정.github.io 검색 : FrontEnd 수업할 때 정적페이지 배포하면서 모두 만들었음. > 혹시 없으면 본인계정.github.io 이름으로 프로젝트 새로 생성하기
3	<u>git 에 올리기</u> <u>본인 깃에 올려서</u> <u>URL이 필요</u>	https://8253jang.github.io/react-data/veggie2.json https://8253jang.github.io/react-data/veggie3.json

10. App.jsx에서 더보기 버튼 누르면 상품을 3개씩 가져오기 (비동기통신 Axios)

순서	작업환경	명령어
1	터미널 열어서 axios 설치	npm install axios
2	App.jsx 상단에	import axios from 'axios'
3	App.js 리턴위에	let [count, setCount] = useState(1);
4	+3개 더 보기 버튼 안에 onClick 버튼1번 클릭하면 3개 상품이 보이고 버튼2번 클릭하면 3개 상품이 또 보이고 버튼3번 클릭하면 alert창으로 더 이상 상품이 없다고 나옴 - 콘솔창에서 버튼을 누르면 데이터가 오는지 확인하면서 작업하기	<Button variant="outline-success"> + 3개 상품 더 보기 </Button>{' '}</p> 이코드를 수정 <pre> <Button variant="outline-success" count = {count} onClick={() => { if(count==1){ axios.get('https://8253jang.github.io/react-data/veggie2.json').then((result)=>{ let copy10 =[...veggie, ...result.data]; setVeggie(copy10); setCount(count + 1); })else if(count==2){ axios.get('https://8253jang.github.io/react-data/veggie3.json').then((result)=>{ let copy11 =[...veggie, ...result.data]; setVeggie(copy11); setCount(count + 1); }) } if(count===3){ alert("더이상 상품이 없습니다."); } }}> + 3개 상품 더 보기 </Button>{' '} </pre>
		크롬 브라우저에서 3개 상품 더보기 버튼 클릭해보기 (axios로 json상품이 요청되어서 화면에 렌더링되어야 함)

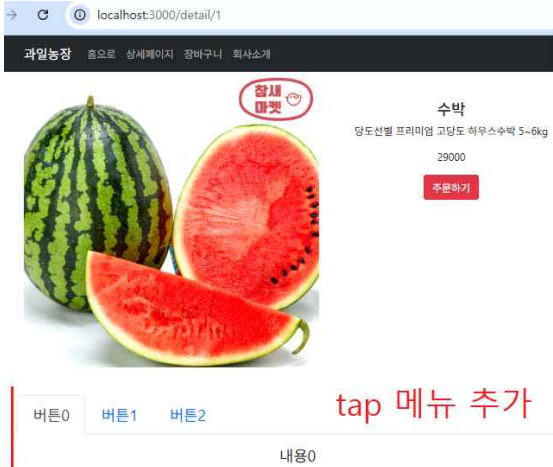
브라우저에서 확인

버튼을 클릭할때마다 json파일을 axios(비동기통신)로 가져와서 렌더링

3번째 클릭하면 상품이 없다고 경고창이 나옴



11 상세페이지 하단 sub 탭 UI 만들기

순서	작업환경	명령어
1	Detail.jsx	상단 import { Nav } from 'react-bootstrap' import { useState } from 'react'; return 앞에 const [tap, setTap] = useState(0); return <div className="container"> 안에</div> 추가코드 작성 <pre><Nav variant="tabs" defaultActiveKey="link0" style={{marginTop:"50px"}}> <Nav.Item> <Nav.Link onClick={()=>{ setTap(0) }} eventKey="link0">버튼0</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link onClick={()=>{ setTap(1) }} eventKey="link1">버튼1</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link onClick={()=>{ setTap(2) }} eventKey="link2">버튼2</Nav.Link> </Nav.Item> </Nav> <TabContent tap={tap}/></pre> 

		전체 코드 참고 <pre> import React from 'react'; import { useParams } from "react-router-dom"; import { Nav } from 'react-bootstrap' import { useState } from 'react'; function Detail(props) { let { paramId } = useParams();// paramId는 문자열로 오기 때문에 숫자로 바꿔줘야 한다. let [tap, setTap] = useState(0); // console.log(paramId); // const { imgUrl, title, content, price } = props.fruit[paramId]; // props.fruit 배열에서 id가 paramId와 일치하는 상품을 찾아 item 변수에 저장 // paramId는 문자열이므로 parseInt를 사용해 숫자로 변환 let item = props.fruit.find(f => f.id === parseInt(paramId)); const { imgUrl, title, content, price } = item; return (<div className="container"> <div className="row"> <div className="col-md-6"> </div> <div className="col-md-6"> <h4 className="pt-5">{title}</h4> <p>{content}</p> <p>{price}</p> <button className="btn btn-danger">주문하기</button> </div> </div> <Nav variant="tabs" defaultActiveKey="link0" style={{marginTop:"50px"}}> <Nav.Item> <Nav.Link onClick={()=>{ setTap(0) }} eventKey="link0">버튼0</Nav.Link> </Nav.Item> <Nav.Item> </pre>
--	--	---

	<pre> <Nav.Link onClick={()=>{ setTap(1) }} eventKey="link1">버튼1</Nav.Link> </Nav.Item> <Nav.Item> <Nav.Link onClick={()=>{ setTap(2) }} eventKey="link2">버튼2</Nav.Link> </Nav.Item> </Nav> <TabContent tap={tap}/> </div>) } function TabContent({tap}){ return [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][tap] } export default Detail;</pre>
export default Detail; 위에 / 1,2,3 다 됨 (동일한 코드)	
1	<pre>function TabContent(props){ if (props.tap === 0){ return <div>내용0</div> }else if (props.tap === 1){ return <div>내용1</div> }else if (props.tap === 2){ return <div>내용2</div> } }</pre>
2	<pre>function TabContent(props){ return [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][props.tap] }</pre> props.tap을 인덱스로 사용하여 배열에서 해당하는 요소를 선택. 선택된 요소는 함수의 반환값으로 사용 props라는 객체에서 tap이라는 속성만 꺼내 쓰는 것이니까, 구조 분해 할당
3	<pre>function TabContent({tap}){ return [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][tap] }</pre> tap을 인덱스로 사용하여 배열에서 해당하는 요소를 선택. 선택된 요소는 함수의 반환값으로 사용 TabContent 함수는 tap에 따라 배열에서 해당하는 요소를 선택하여 렌더링함. const tap = props.tap; 줄여쓴것

브라우저에서 확인	과일농장 홈으로 상세페이지 장바구니 회사소개  수박 당도선별 프리미엄 고당도 하우스수박 5~6kg 29000 주문하기 버튼0 버튼1 버튼2 멀티탭 버튼 클릭 내용1	

참고2 useeffect – 서버화면 애니메이션하기전에 익혀야할 개념

w3s 참고	https://www.w3schools.com/react/react_useeffect.asp		
useEffect <u>렌더링이 될 때마다 어떤 작업을 진행시키고 싶을 경우</u> <u>전체 페이지가 다 로딩되고 나서 나중에 실행하는 함수</u> sideEffect처럼 주요기능 이외의 함수를 뜻함 setTimeout, ajax 등 페이지가 다 로딩(렌더링)되고 나서 나중에 일하는 함수를 useEffect에 넣어 사용	상단 import	import { useEffect } from "react";	
	함수 리턴위에	useEffect()=>{ //여기적은 코드는 컴포넌트 로드 & 업데이트 마다 실행됨 };	
	렌더링되고 딱 한번만 실행될때	tap 의 state가 바뀔때마다 실행	렌더링 이전 에 대한 정보들을 확인하고 싶다면 return 을 사용(자원해지)
	useEffect() => { //코드 }, []);	useEffect() => { //코드 }, [tap]);	useEffect()=>{ //코드 return ()=>{ setFade("") } } ,[tap])

12 상세페이지 클릭시, 하단 sub 탭 UI 클릭 시 애니메이션(Transition) 넣기

순서	작업환경	명령어						
1	Detail.jsx							
	애니메이션 만들고 싶으면 ① 애니메이션 동작 전 스타일을 담은 className 만들기 ② 애니메이션 동작 후 스타일을 담은 className 만들기 ③ transition 속성도 추가 ④ 원할 때 ②번 탈부착	index.css 요소의 크기가 0과 1로 커지는 scalecss Detail.jsx 상단에 Detail.jsx export default Detail; 위에 위의 코드 다시 수정 return위의 코드 추가 <table><tr><td>빨강색</td><td>useEffect</td></tr><tr><td>파란색</td><td>tap이 바뀌면(렌더링되고) 나서의 실행코드</td></tr><tr><td>녹색</td><td>렌더링 이전의 정보 확인 리턴사용</td></tr></table> tap을 누를때마다 빈문자열로 만들고 0.01초 후에 end글씨로 바꿈 start : 0 fade : "" 빈문자열, 클릭후 fade : end 글자가 됨	빨강색	useEffect	파란색	tap이 바뀌면(렌더링되고) 나서의 실행코드	녹색	렌더링 이전의 정보 확인 리턴사용
	빨강색	useEffect						
	파란색	tap이 바뀌면(렌더링되고) 나서의 실행코드						
녹색	렌더링 이전의 정보 확인 리턴사용							
		.start { transform: scale(0); } .end { transform: scale(1); transition :all 0.5s; } import { useEffect } from "react"; function TabContent({tap}){ return (<div className="start end"> { [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][tap] } </div>) } function TabContent({tap}){ let [fade, setFade] = useState("") useEffect(()=>>{ setTimeout(0=>{ setFade('end')},10) return ()=>>{ setFade("") } } ,[tap]) return (<div className={`start ` + fade}> { [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][tap] } </div>) }						

위의 코드 설명 (F12 눌러서 개발자 도구에서 확인 - 요소확인)

① className="start end"

② useEffect 실행 전엔 'end'가 ' '

③ automatic batching
마지막에 한 번만 재렌더링 되므로
시간차를 두기 위해
setTimeout사용

 개념 설명: Automatic Batching

 상태 업데이트란?

setState() 나 useState() 의 setXXX() 함수로 상태를 변경할 때마다 React는 리렌더링을 발생시킵니다.

 Batching이란?

여러 개의 상태 변경이 일어날 때, 작가를 하나씩 처리하면 리렌더링이 여러 번 발생합니다.
Batching은 이 상태 변경들을 하나의 그룹(batch) 으로 묶어서 리렌더링을 한 번만 하도록 해줍니다.

 Automatic Batching이란?

React가 자동으로 상태 업데이트를 묶어주는 기능입니다.

1. 이제 탭이라는 state가 변할 때마다 .start와 .end 스타일을 둘 다 적용하겠다

2. 'start ' + fade

fade의 초기값은 " (빈 문자열)

나중에 setFade('end')이 되면 'start ' + 'end'가 돼서 className="start end"

기본적으로 'start' 클래스는 항상 들어가고, fade의 값이 바뀌면서 'end' 클래스가 추가로 붙는 구조

3. automatic batching 한 번만 재렌더링 되므로 시간차를 두기 위해
setTimeout사용

리액트 18부터는 automatic batching으로
여러 setState가 동시에 실행되면 한 번만 리렌더링된다.
그래서 ' ' → 'end'로 바로 바꾸면
클래스 변경이 감지되지 않아 애니메이션이 안 된다.
그래서 setTimeout으로 시간차를 줘서 리렌더링을 강제한다.

버튼 누르면 → 바로 fade는 " 되고
약간 있다가 → fade가 'end' 된다.

html에서는 css name호출

<div class="start fade"> </div>

React에서 className 호출

<div className={ 'start ' + fade}> </div>

2

효과 둘중 하나만 (위의예제는 scale)
Detail 컴포넌트 로드시 투명도가 0에서 1로
서서히 증가하는 애니메이션을 주려면?

Detail.jsx

Detail.jsx return 위	<pre>let [fade2, setFade2] = useState("") useEffect(()=>{ setFade2('end') return ()=>{ setFade2("") } },[])</pre>
return 아래	<pre><div className={`container start ` + fade2}></pre>
Detail.jsx 전체코드	<pre>import React from 'react'; import { useParams } from "react-router-dom"; import { Nav } from 'react-bootstrap' import { useState } from 'react'; import { useEffect } from "react"; function Detail(props) { let { paramId } = useParams();// paramId는 문자열로 오기 때문에 숫자로 바꿔줘야 한다. let [tap, setTap] = useState(0); // props.fruit 배열에서 id가 paramId와 일치하는 상품을 찾아 item 변수에 저장 // paramId는 문자열이므로 parseInt를 사용해 숫자로 변환 let item = props.fruit.find(f => f.id === parseInt(paramId)); const { imgUrl, title, content, price } = item; let [fade2, setFade2] = useState("") useEffect(()=>{ setFade2('end') return ()=>{ setFade2("") } })</pre>

},0)

```
return (  
  <div className={`container start ` + fade2}>  
    <div className="row">  
      <div className="col-md-6">  
        <img src={`/` + imgUrl} width="100%" alt={title} />  
      </div>  
      <div className="col-md-6">  
        <h4 className="pt-5">{title}</h4>  
        <p>{content}</p>  
        <p>{price}</p>  
        <button className="btn btn-danger">주문하기</button>  
      </div>  
    </div>  
  
    <Nav variant="tabs" defaultActiveKey="link0" style={{marginTop:"50px"}}>  
      <Nav.Item>  
        <Nav.Link onClick={()=>{ setTap(0) }} eventKey="link0">버튼0</Nav.Link>  
      </Nav.Item>  
      <Nav.Item>  
        <Nav.Link onClick={()=>{ setTap(1) }} eventKey="link1">버튼1</Nav.Link>  
      </Nav.Item>  
      <Nav.Item>  
        <Nav.Link onClick={()=>{ setTap(2) }} eventKey="link2">버튼2</Nav.Link>  
      </Nav.Item>  
    </Nav>  
  
    <TabContent tap={tap}/>  
  </div>  
)  
}
```

```
function TabContent({tap}){  
  let [fade, setFade] = useState("")  
  useEffect( ()=>{  
    setTimeout(()=>{ setFade('end')},10)
```

		<pre> return ()=>{ setFade("") } },[tap]) return (<div className={'start ' + fade}> { [<div>내용0</div>, <div>내용1</div>, <div>내용2</div>][tap] } </div>) } export default Detail;</pre>
--	--	--

브라우저에서 확인

F12 눌러서 개발자모드에서 확인
멀티탭에서 확인 버튼 클릭시 start+end

상세페이지에서 확인
버튼 클릭시 start+end
상세페이지가 렌더링될 때마다 상품상세페이지가 작아졌다가 커지는지 확인



버튼0 버튼1 버튼2

내용1

수박
당도선별 프리미엄 고당도 하우스수박 5~6kg
29000
주문하기

```
<!DOCTYPE html>
<html lang="en">
  <head>=</head>
  <body>
    <script>You need to enable JavaScript to run this app.</script>
    <div id="root">
      <div class="App">
        <nav class="navbar navbar-expand navbar-dark bg-dark">=</nav>
        <div class="container start end">
          <div class="row">=</div>
          <div class="nav-item">=</div>
          <div class="nav-item">=</div>
          <a href="#">버튼1</a>=</div>
        </div>
        <div class="nav-item">=</div>
        <div class="start end">=</div>
      </div>
    </div>
  </body>
</html>
```

This HTML file is a template.
If you open it directly in the browser, you will see an empty page.
You can add webfonts, meta tags, or analytics to this file.
The build step will place the bundled scripts into the <body> tag.
To begin the development, run "npm start" or "yarn start".
To create a production bundle, use "npm run build" or "yarn build".

과일농장

홈으로

상세페이지

장바구니

회사소개

참새마켓

수박

당도선별 프리미엄 고당도 하우스수박 5~6kg

29000

주문하기

Elements

Console

Sources

Network

Perf

F12

<script>You need to enable JavaScript to run this app.</script>

<div id="root">

<div class="App">

<nav class="navbar navbar-expand navbar-dark bg-dark">

<div class="container start end">

<div class="row">

<div class="nav-item">

<div class="nav-item">

버튼1

네비바에서

회사소개 눌렀다가

상세페이지오면

콘솔창에서

start 0.01초만에

end가 붙는다

(크기가 1로 확대)